

Files Analysed

- `endpoint.py`
- `trace_demo.py`

Contents

1	Executive Summary	2
2	File 1: endpoint.py	3
2.1	Intent and Context	3
2.2	Inputs	3
2.3	Algorithm	3
2.4	Implementation Walkthrough	3
2.5	Output	4
2.6	Design Observations and Limitations	4
3	File 2: trace_demo.py	6
3.1	Intent and Context	6
3.2	The Subject Computation	6
3.3	The Four Tracing Methods	6
3.3.1	Method 1: Built-in <code>trace</code> Module	6
3.3.2	Method 2: <code>sys.settrace</code> (Low-level)	6
3.3.3	Method 3: Decorator / Monkey-Patch Approach	7
3.3.4	Method 4: viztracer (External Library)	7
3.4	Summary Block	8
4	Relationship Between the Two Files	9
5	Methodology Comparison Table	10
6	Key Findings and Recommendations	10
6.1	Strengths	10
6.2	Areas for Improvement	10
7	Conclusion	11

1 Executive Summary

The two files under review form a complementary pair of diagnostic and analysis utilities built around the **mpyc** Python library — a framework for secure multi-party computation (MPC). Together they address the question:

Goal

“Which low-level functions in mpyc are truly foundational (i.e. not called by any other internal function), and which internal functions are actually exercised when a real MPC operation runs?”

- `endpoint.py` is a *post-processing* script. It reads two pre-generated artefact files — a list of functions required for a planned C port of mpyc, and a call-dependency trace — and computes their set difference to isolate “endpoint” functions: those which are required but never called by any other internal function.
- `trace_demo.py` is a *live instrumentation* script. It exercises a minimal mpyc computation and captures which internal functions are invoked, using four distinct tracing strategies of increasing sophistication.

These tools are clearly part of a larger engineering effort to port mpyc’s computational kernel to native C, where understanding the dependency graph and identifying atomic building blocks is a prerequisite.

2 File 1: endpoint.py

2.1 Intent and Context

The filename and the output file it creates (`mpyc.Cnative_endpoints.txt`) reveal the broader project context: a team is porting a subset of mpyc functionality to C. To do this correctly, they first need to know:

1. Which functions are *required* for the C port (`mpyc.Cnative.txt`).
2. Which functions *call other internal functions* (`mpyc_dependency_trace.txt`).
3. By exclusion — which required functions are *leaf nodes* in the call graph, i.e. they call nothing else internally.

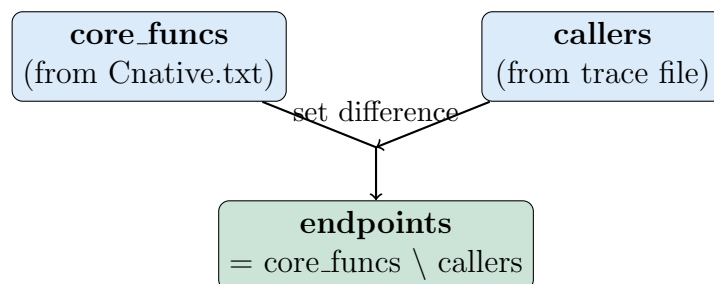
These leaf-node functions are what the script terms “**endpoints**”: the independent building blocks that can be ported to C first, without any internal dependency chain.

2.2 Inputs

File	Description
<code>mpyc.Cnative.txt</code>	A list of function names required for the native C implementation. The first 4 lines are skipped (headers/metadata), and each subsequent non-empty line is treated as one function name.
<code>mpyc_dependency_trace.txt</code>	A call-dependency trace where each relevant line has the format: <code>callerFunc ---> calls ---> calleeFunc</code> . Only the caller (left side) is extracted.

2.3 Algorithm

The core logic is a simple set-difference operation:



Formally:

$$\text{endpoints} = \{f \in \text{core_funcs} \mid f \notin \text{callers}\}$$

A function is an endpoint if and only if (a) it appears in the C-required list AND (b) it was never observed *calling* another function in the dependency trace.

2.4 Implementation Walkthrough

```

1 # Step 1: Parse the C-native function list
2 core_funcs = set()
3 with open(cnative_file, "r", encoding="utf-8") as f:

```

```
4     lines = f.readlines()
5     for line in lines[4:]:          # skip 4-line header block
6         func_name = line.strip()
7         if func_name:
8             core_funcs.add(func_name)
9
10    # Step 2: Extract all callers from the trace
11    callers = set()
12    with open(trace_file, "r", encoding="utf-8") as f:
13        for line in f:
14            if "---> calls --->" in line:
15                caller = line.split("---> calls --->")[0].strip()
16                callers.add(caller)
17
18    # Step 3: Set difference
19    endpoints = sorted([func for func in core_funcs if func not in
        callers])
```

Listing 1: Core set operations in endpoint.py

2.5 Output

The script produces both console output and a persistent file `mpyc_Cnative_endpoints.txt`. The output file begins with a 5-line header describing the contents and the total count, followed by one function name per line.

Output format

INDEPENDENT ENDPOINT FUNCTIONS FOR NATIVE C PORT

=====

These are the lowest-level building blocks. They are required for the core C library, but do not depend on any other internal mpyc functions. (Total: N)

func_name_1

func_name_2

...

2.6 Design Observations and Limitations

- **Stateless / single-pass:** The script performs no graph traversal or transitive closure. It only looks one level deep — whether a function *directly* calls others.
- **Hardcoded delimiter:** The trace file parser relies on the exact string `"---> calls --->"`, implying a specific format contract with the upstream trace generator.
- **Header skip is fragile:** Skipping exactly 4 lines assumes the upstream `mpyc_Cnative.txt` always has a 4-line header. A format change would silently corrupt results.
- **No validation:** There is no check that the two input files share a consistent function namespace, meaning mismatches (e.g., due to aliasing) would produce silent false positives.

- **Good error handling:** The script does check for missing files and prints actionable error messages rather than crashing.

3 File 2: trace_demo.py

3.1 Intent and Context

This script is explicitly annotated as being prepared “*For Chris presentation*” — it is a demonstration and teaching artefact. Its purpose is to showcase four different mechanisms in Python for tracing function calls, using a real mpyc computation as the subject. The computation itself is deliberately simple (adding 3 and 4 under MPC) so attention stays on the *tracing methodology*, not the cryptographic protocol.

3.2 The Subject Computation

```
1 async def run_mpc():
2     from mpyc.runtime import mpc
3     async with mpc:
4         secint = mpc.SecInt()           # define a secure integer
5                                         type
6         a = mpc.input(secint(3), senders=0) # secret-share the
7                                         value 3
8         b = mpc.input(secint(4), senders=0) # secret-share the
9                                         value 4
10        c = mpc.add(a, b)                # homomorphic addition
11        result = await mpc.output(c)      # reveal the result
12        return result
```

Listing 2: The MPC computation being traced

This is the idiomatic “hello world” of MPC: secret-sharing two values and computing their sum securely. It invokes the key lifecycle operations — `input`, `add`, `output` — making it a good exerciser of the internal call graph.

3.3 The Four Tracing Methods

3.3.1 Method 1: Built-in trace Module

Mechanism: Uses Python’s standard library `trace.Trace` class with the `countfuncs=True` flag, which records every unique (file, module, function) triple that was called during execution.

Pros: Zero external dependencies; simple to invoke.

Cons: Very coarse — provides presence/absence, not call sequence or frequency. The internal data structure `tracer._calledfuncs` is a private attribute (underscore-prefixed), implying mild API fragility.

Output: A filtered list of all unique mpyc functions called, formatted as `funcname [mpyc.module]`.

3.3.2 Method 2: `sys.settrace` (Low-level)

Mechanism: Installs a custom frame-level event hook via `sys.settrace`. The hook is triggered on every “call” event, at which point the module name, function name, and source line number are captured. After execution, duplicates are removed while preserving call order.

Pros: Fine-grained control; captures call sequence; no external dependencies.

Cons: Manual setup and teardown (try/finally ensures cleanup); can have performance overhead on large codebases.

```
1 def tracer(frame, event, arg):
2     if event == "call":
3         module = frame.f_globals.get("__name__", "?")
4         func    = frame.f_code.co_name
5         lineno  = frame.f_lineno
6         if "mpyc" in module:
7             call_log.append(f"{module}.{func} (line {lineno})")
8     return tracer    # recursive: keep tracing into callees
```

Listing 3: sys.settrace callback

Output: A deduplicated, order-preserving call sequence. This output is reused later in the SUMMARY section to filter for the most essential functions.

3.3.3 Method 3: Decorator / Monkey-Patch Approach

Mechanism: Rather than installing a global trace hook, this method surgically wraps specific known methods on the mpc singleton object using `functools.wraps`. A hardcoded list of 14 target methods (e.g. `input`, `output`, `add`, `mul`) is patched before the computation runs, and restored afterwards.

```
1 def make_wrapper(name, fn):
2     @functools.wraps(fn)
3     def wrapper(*args, **kwargs):
4         call_counts[name] = call_counts.get(name, 0) + 1
5         return fn(*args, **kwargs)
6     return wrapper
```

Listing 4: Monkey-patch wrapper factory

Pros: Clean, targeted output; minimal overhead; readable call counts per method.

Cons: Only covers the explicitly listed methods — any unlisted internal helpers are invisible. Requires knowing the API surface in advance.

Output: A call-count table sorted in descending frequency, showing how many times each patched method was invoked.

3.3.4 Method 4: viztracer (External Library)

Mechanism: Uses the optional third-party `viztracer` library to produce a full flamegraph / timeline trace saved to `mpc_trace.json`, viewable interactively via `vizviewer`.

Pros: Rich visual output; timeline view; can inspect async coroutine interleaving.

Cons: Requires installation (`pip install viztracer`); handled gracefully with a try/except that degrades to a helpful install message.

Output: A JSON artefact for interactive visual inspection.

3.4 Summary Block

After all four methods execute, the script performs a final filter on the `sys.settrace` call log to surface only the functions most directly related to the core MPC operation:

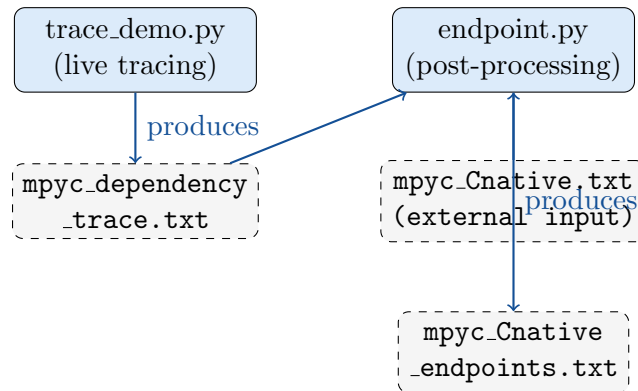
```
1 required = [c for c in calls
2             if any(k in c for k in
3                   ["input", "output", "add", "_mpc", "SecInt", "
                     _sec"])]
```

Listing 5: Post-run essential function filter

This heuristic keyword filter is a quick approximation — not a formal dependency analysis. It surfaces the handful of calls that are clearly semantically central to the computation.

4 Relationship Between the Two Files

The two scripts sit at different stages of the same investigation pipeline:



- `trace_demo.py` (or a more comprehensive variant of it) would generate the `mpyc_dependency_trace.txt` consumed by `endpoint.py`.
- `endpoint.py` then cross-references that trace against the C-port target list to find the atomic, independently portable functions.
- The output of `endpoint.py` directly informs *where to start* the C port: implement the endpoints first, then build upward through the call graph.

5 Methodology Comparison Table

Method	Dependencies	Granularity	Best Use
<code>trace module</code>	None (stdlib)	Function presence	Quick inventory
<code>sys.settrace</code>	None (stdlib)	Call sequence + line numbers	Order-sensitive analysis
Monkey-patch	None (stdlib)	Call counts on target methods	Targeted profiling
viztracer	<code>pip install</code>	Full flamegraph timeline	Deep visual debugging

Aspect	<code>endpoint.py</code>	<code>trace_demo.py</code>
Purpose	Static post-analysis	Live instrumentation
Input	Text files	Running mpyc process
Technique	Set arithmetic	Frame hooks / wrappers
Output	Endpoint function list	Call logs / counts / flamegraph
Audience	C port engineer	Presenter / developer audience

6 Key Findings and Recommendations

6.1 Strengths

- Both scripts are clean and readable, with clear docstrings and section comments.
- Error handling in `endpoint.py` is thoughtful (file existence checks, actionable messages).
- `trace_demo.py` uses `functools.wraps` correctly in the monkey-patch method, preserving function metadata through the wrapper.
- The `try/finally` block in Method 2 ensures the global trace hook is always uninstalled, preventing interference with other code.
- Graceful degradation in Method 4 (`try/except ImportError`) is good practice.

6.2 Areas for Improvement

- **Header skip fragility:** The `lines[4:]` slice in `endpoint.py` is a brittle convention. A more robust approach would use a sentinel line (e.g. a blank line or a `---` separator) to mark where the function list begins.
- **Private API access:** Accessing `tracer._calledfuncs` in Method 1 relies on an internal attribute. Using `tracer.results().calledfuncs` (the public API equivalent) would be more stable.
- **Transitive dependencies:** `endpoint.py` only identifies direct-call leaf nodes. For a complete C-port dependency graph, a transitive closure (e.g. via BFS/DFS on the trace graph) would give the full ordering of what to port in which sequence.

- **Async handling:** The `sys.settrace` approach does not natively handle async coroutine interleaving. For deep async analysis, `sys.monitoring` (Python 3.12+) or `viztracer` is preferable.
- **Namespace consistency:** `endpoint.py` has no cross-validation between the two input files, which could produce incorrect results if function names are represented differently (e.g. with or without module prefixes).

7 Conclusion

These two scripts represent a well-structured, pragmatic approach to reverse-engineering the internal call topology of the mpyc library in preparation for a selective C port. `trace_demo.py` provides a flexible, multi-method toolkit for live introspection, while `endpoint.py` applies graph-theoretic reasoning (set difference on a call graph) to identify the foundational functions that form the natural starting point for native re-implementation.

Summary in one sentence

Together, the scripts answer: *“Of all the mpyc functions we need to port to C, which ones have no internal dependencies and can therefore be implemented first and in isolation?”*